(一) prompt 記錄 (順序:從左到右)

"用 Python + numpy（不使用 sklearn 或 PyTorch 等 ML 框架），
從零實作梯度下降法訓練線性回歸 y = wx + b，
資料：x=[166,176,171,173,169], y=[58,75,62,70,60]
設定：learning_rate=0.0001, epochs=2000
要求：
1. 隨機初始化 w=0, b=0
2. 每次迭代計算 MSE 和梯度 dL/dw、dL/db
3. 用梯度更新 w, b（寫出更新公式的程式碼）
4. 每 400 次迭代印出 epoch、MSE、w、b
5. 繪製 Loss 隨 epoch 下降的收斂曲線，標示最終 w*, b*
給我一點提示

我先不直接把完整答案丟給你，給你剛好能自己完成的關鍵提示（這種其實很經典 🔥）：

🧠 1. 先想清楚模型

你的線性回歸就是：

$$y = wx + b$$

用 numpy 表示就是：

```python
y_pred = w * x + b
```

📉 2. Loss（MSE）怎麼寫？

MSE 公式：

$$L = \frac{1}{n}\sum(y_{pred} - y)^2$$

👉 提示：

```python
loss = np.mean((y_pred - y) ** 2)
```

🔍 3. 梯度怎麼來（最重要🔥）

你要自己寫這兩個：

$$\frac{dL}{dw} = \frac{2}{n}\sum(y_{pred} - y)\cdot x$$

$$\frac{dL}{db} = \frac{2}{n}\sum(y_{pred} - y)$$

👉 對應 numpy：

```python
dw = (2/n) * np.sum((y_pred - y) * x)
db = (2/n) * np.sum(y_pred - y)
```

➡️ 4. 梯度下降更新（核心）

這一步你一定要寫對：

```python
w = w - learning_rate * dw
b = b - learning_rate * db
```

🔁 5. 訓練流程骨架

提示你 loop 應該長這樣：

```python
for epoch in range(epochs):
    # 1. 預測
    # 2. 算 loss
    # 3. 算梯度
    # 4. 更新參數

    if epoch % 400 == 0:
        print(...)
```

🧠 先改你的架構（關鍵）

👉 把訓練包成 function：

```python
def train(method):
    w, b = 0, 0
    state = {}
    loss_history = []

    for i in range(2000):
        y_pred = w * x + b
        loss = np.mean((y_pred - y_true)**2)

        dw = (2/n)*np.sum((y_pred - y_true)*x)
        db = (2/n)*np.sum(y_pred - y_true)
```

1 SGD

```python
if method == "sgd":
    w -= learning_rate * dw
    b -= learning_rate * db
```

2 Momentum

```python
if method == "momentum":
    beta = 0.9

    v_w = state.get("v_w", 0)
    v_b = state.get("v_b", 0)

    v_w = beta * v_w + (1-beta) * dw
    v_b = beta * v_b + (1-beta) * db

    w -= learning_rate * v_w
    b -= learning_rate * v_b

    state["v_w"] = v_w
    state["v_b"] = v_b
```

4 RMSProp

```python
if method == "rmsprop":
    beta = 0.9

    G_w = state.get("G_w", 0)
    G_b = state.get("G_b", 0)

    G_w = beta * G_w + (1-beta) * dw**2
    G_b = beta * G_b + (1-beta) * db**2

    w -= learning_rate * dw / (np.sqrt(G_w) + 1e-8)
    b -= learning_rate * db / (np.sqrt(G_b) + 1e-8)

    state["G_w"] = G_w
    state["G_b"] = G_b
```

5 Adam（最重要🔥）

```python
m_w = state.get("m_w", 0)
v_w = state.get("v_w", 0)
m_b = state.get("m_b", 0)
v_b = state.get("v_b", 0)

m_w = beta1 * m_w + (1-beta1) * dw
v_w = beta2 * v_w + (1-beta2) * dw**2

m_b = beta1 * m_b + (1-beta1) * db
v_b = beta2 * v_b + (1-beta2) * db**2

w -= learning_rate * m_w / (np.sqrt(v_w) + 1e-8)
b -= learning_rate * m_b / (np.sqrt(v_b) + 1e-8)

state["m_w"] = m_w
state["v_w"] = v_w
state["m_b"] = m_b
state["v_b"] = v_b
```

他有五種優化器

很好，你現在差最後一步了 👍
老師說「五種優化器」，通常就是這五個👇

🎯 常見五種優化器（你要實作的）

1. SGD（Batch GD）
2. Momentum
3. AdaGrad
4. RMSProp
5. Adam

🔥 重點：你只要補「更新規則」就完成

我幫你整理成可以直接塞進你程式的版本（最小修改）

📊 6. Loss 曲線（收斂圖）

👉 提示：你需要一個 list

Python 執行
```python
loss_history = []
```

每次：

Python 執行
```python
loss_history.append(loss)
```

畫圖：

Python 執行
```python
plt.plot(loss_history)
```

⭐ 7. 最後標註 w*, b*

👉 提示：

Python 執行
```python
plt.title(f"Final w={w:.2f}, b={b:.2f}")
```

⚠️ 8. 小心這幾個坑（超重要）

1. x, y 要轉 numpy array

Python 執行
```python
x = np.array([...])
y = np.array([...])
```

2. n 是資料數：

Python 執行
```python
n = len(x)
```

3. learning_rate 太大會爆炸（你這邊已經幫你設好了 👍）

但是我b的變數要如何設定

很好，這個問題其實是很多人一開始會卡的點 👍

🎯 b 到底是什麼？

在模型

$$y = wx + b$$

👉 b 就是截距（bias），本質上只是「一個數字」。

🧩 1. 初始化 b（超簡單）

題目已經指定：

> 隨機初始化 w=0, b=0

所以你直接寫：

Python 執行
```python
w = 0.0
b = 0.0
```

👉 用 `0.0`（float）比較安全，避免整數問題

🔁 2. b 在訓練中怎麼用？

每次預測：

Python 執行
```python
y_pred = w * x + b
```

👉 numpy 會自動把 `b` 加到每個元素（broadcasting）

🔍 3. b 的梯度（你真正卡的地方）

你只要記住：

$$\frac{dL}{db} = \frac{2}{n}\sum(y_{pred} - y)$$

對應程式：

Python 執行
```python
db = (2/n) * np.sum(y_pred - y)
```

➡️ 4. 更新 b

Python 執行
```python
b = b - learning_rate * db
```

👉 跟 w 完全一樣，只是沒有乘 x

🧠 直覺理解（幫你真正搞懂）

- w：控制斜率（線多陡）
- b：控制整條線上下移動

👉 如果你的預測整體偏高 → b 會下降
👉 如果整體偏低 → b 會上升

我這樣是對的嗎?

```python
import numpy as np
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
import matplotlib.pyplot as plt

plt.rcParams['font.family'] = 'Microsoft JhengHei' #中文
plt.rcParams['axes.unicode_minus'] = False  #負號

x = np.array([166,176,171,173,169], dtype=float)
y_true = np.array([58,75,62,70,60], dtype=float)


w=0
b=0
n=5
learning_rate=0.001
loss_history = []

linear1_line = w * x + b #線性回歸模型
plt.plot(x, linear1_line, label="y=0.5x")

loss =np.mean((w*x+b-y_true)**2) #numpy MSE公式
print("w:", w)
print("b:", b)
print("loss:", loss)

dw=(2/n)*np.sum((w*x+b-y_true)*x)
db=(2/n)*np.sum(w*x+b-y_true) #numpy梯度公式
w= w - learning_rate*dw      #梯度下降更新
b= b - learning_rate*db

for i in range(2000):
    dw=(2/n)*np.sum((w*x+b-y_true)*x)
    db=(2/n)*np.sum(w*x+b-y_true) #numpy梯度公式
    w= w - learning_rate*dw      #梯度下降更新
    b= b - learning_rate*db
    loss_history.append(loss)

plt.plot(loss_history, label="損失")
```

🔴 2. learning rate 跟題目不一致

題目給的是：

```python
learning_rate = 0.0001
```

你寫：

```python
learning_rate = 0.001
```

👉 這可能會：

- 收斂太快（甚至震盪）
- 或結果不穩定

🟡 3. n 不要寫死（好習慣）

你寫：

```python
n = 5
```

👉 建議改成：

```python
n = len(x)
```

（之後換資料才不會爆）

🟢 4. 初始畫線是錯的（小問題）

```python
linear1_line = w * x + b
plt.plot(x, linear1_line, label="y=0.5x")
```

👉 但你實際上是：

- w = 0
- b = 0

所以這條線是 y = 0
但 label 寫 "y=0.5x" ❌

🔴 1. 最大問題：loss 沒有更新（❗很重要）

你現在是：

```python
loss = np.mean((w*x+b-y_true)**2)
```

但在 for 迴圈裡你沒有重新計算 loss！

```python
loss_history.append(loss)
```

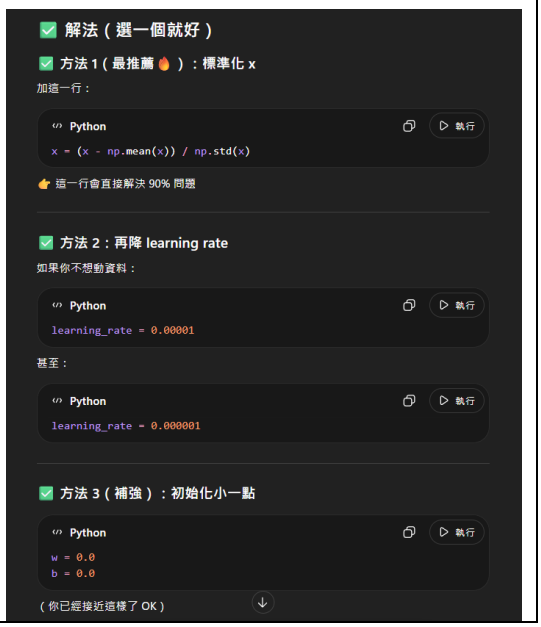
👉 這樣會變成：
2000 次都在存「同一個 loss」→ 圖會是一條直線 😵

✅ 正確做法（提示）：

```python
for i in range(2000):
    y_pred = w*x + b

    loss = np.mean((y_pred - y_true)**2)   # ⭐ 每次都要更新

    dw = ...
    db = ...

    w = ...
    b = ...

    loss_history.append(loss)
```

(二) 遇到的問題用 AI 改善

```
  w= w - learning_rate*dw
i=800, w=nan, b=nan, loss=nan
i=1200, w=nan, b=nan, loss=nan
i=1600, w=nan, b=nan, loss=nan
```

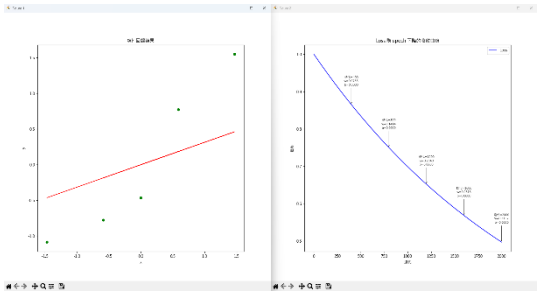


在一開始跑程式遇到 loss 報錯誤，原因是沒有正規化，以至於資料變動太大無法收斂。

(三) 四種學習率訓練 (以 SDG 為例、左圖對應右圖)

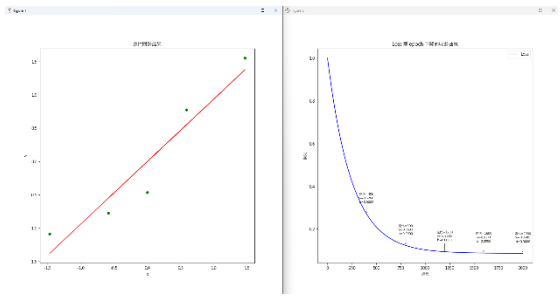
```
(task) c:\Users\Edward\Desktop\人工智慧作業> c: && cd c:\Users\Edward\Desktop\人工智慧作業 &&
debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher 56310 -- C:\Users\Edward\Desktop\人
迭代=400, w=0.0735054206097385 4, b=8.526512829121222e-19, loss=0.8651768864380778
迭代=800, w=0.1413589330199538 6, b=1.7275070263167477e-18, loss=0.7499784560661467
迭代=1200, w=0.2039951183272629 7, b=2.5668356329333455e-18, loss=0.6518143999729625
迭代=1600, w=0.2618151422360162 6, b=3.3706371027619396e-18, loss=0.5681658478394465
迭代=2000, w=0.3151893244025464, b=4.214406601477026e-18, loss=0.4968863943153218 3
```

學習率:0.0001



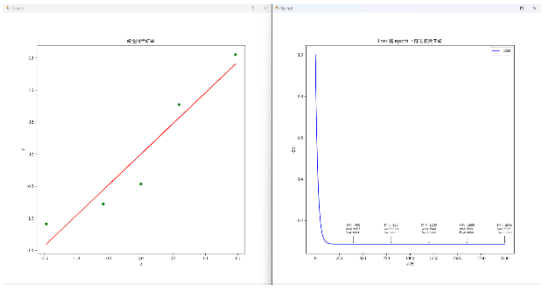
```
迭代=400, w=0.5267681846151817, b=8.282263763703695e-18, loss=0.2710760840759088 3
迭代=800, w=0.7632708566487005, b=1.7164047960705202e-17, loss=0.1234054082336256
迭代=1200, w=0.8694532638490426, b=2.6045832157706 71e-17, loss=0.0936389411390159
迭代=1600, w=0.917125890555185, b=2.77555756156295e-17, loss=0.0876388158044450 7
迭代=2000, w=0.9385294299642644, b=2.77555756156295e-17, loss=0.0864293506732242 6
```

學習率:0.001



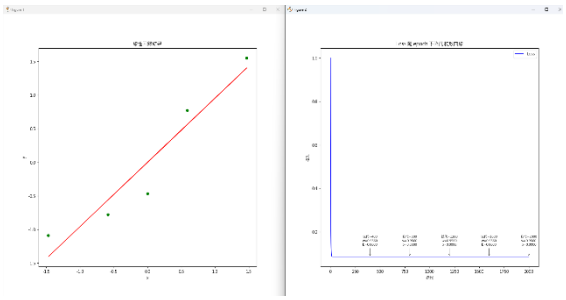
```
迭代=400, w=0.9556729005776139, b=2.775557561562893e-17, loss=0.086124096358336
迭代=800, w=0.955968524481741, b=2.775557561562893e-17, loss=0.08612400530504852
迭代=1200, w=0.9559686159288173, b=2.775557561562893e-17, loss=0.08612400530503979
迭代=1600, w=0.9559686159571051, b=2.775557561562893e-17, loss=0.08612400530503979
迭代=2000, w=0.9559686159571112, b=2.775557561562893e-17, loss=0.0861240053050398
```

學習率:0.01



```
迭代=400, w=0.9559686159571137, b=2.8865798640254083e-17, loss=0.0861240053050398
迭代=800, w=0.9559686159571137, b=2.8865798640254083e-17, loss=0.0861240053050398
迭代=1200, w=0.9559686159571137, b=2.8865798640254083e-17, loss=0.0861240053050398
迭代=1600, w=0.9559686159571137, b=2.8865798640254083e-17, loss=0.0861240053050398
迭代=2000, w=0.9559686159571137, b=2.8865798640254083e-17, loss=0.0861240053050398
```

學習率:0.1



學習率 η	課程描述	你觀察到的 Loss 曲線
0.0001	η 過小	（請填寫觀察）
0.001	接近適中	（請填寫觀察）
0.01	η 適中	（請填寫觀察）
0.1	η 過大	（請填寫觀察）

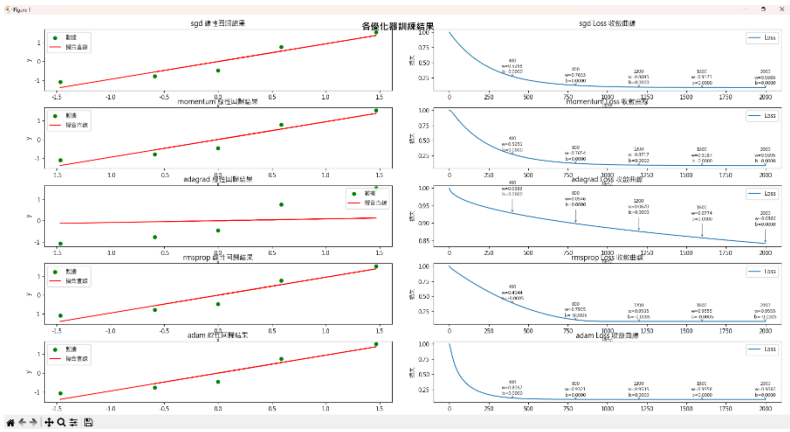
學習率 0.0001:可以發現在回歸曲線中數據點離得相當遠，但是在 Loss 曲線中卻擁有最好的收斂效果，但是因為從回歸曲線中看出並不再資料上訓練，所以效過仍然是不理想。

學習率 0.001:回歸曲線比較接近理想，在 Loss 曲線中也有比較好的收斂效果。

學習率 0.01、學習率 0.1:回歸曲線貼近資料，但是因為 Loss 曲線根本沒有收斂，故訓練效果肯定差。

(四) 五種優化器 Loss 收斂比較 (以學習率 0.001 為例)

```
現在訓練：sgd
迭代=400, w=0.5267681846151817, b=8.282263763703695e-18, loss=0.27107608407590883
迭代=800, w=0.7632708566487005, b=1.7164047960705202e-17, loss=0.1234054082336256
迭代=1200, w=0.8694532638490426, b=2.604583215770671e-17, loss=0.0936389411390159
迭代=1600, w=0.917125890555185, b=2.775557561562 95e-17, loss=0.08763881580444507
迭代=2000, w=0.9385294299642644, b=2.775557561562 95e-17, loss=0.08642935067322426
現在訓練：momentum
迭代=400, w=0.525078228940715, b=7.971736252428941e-18, loss=0.27254941696473833
迭代=800, w=0.7653932508145661, b=1.6853185513809974e-17, loss=0.12259142382946422
迭代=1200, w=0.8716804283029455, b=2.573496971081148e-17, loss=0.09325754253044279
迭代=1600, w=0.9186894084755475, b=2.795541576006184e-17, loss=0.08751942498115776
迭代=2000, w=0.9394806672998742, b=2.795541576006184e-17, loss=0.08639696891495316
現在訓練：adagrad
迭代=400, w=0.03828609490535765, b=2.9335351361940396e-17, loss=0.9283556828539407
迭代=800, w=0.05456195820280042, b=6.98643180192373e-18, loss=0.8987204163709425
迭代=1200, w=0.06698453119311289, b=2.9335351361940 31e-17, loss=0.8764667516098654
迭代=1600, w=0.07741106894857937, b=2.9335351361940 36e-17, loss=0.858030021908268
迭代=2000, w=0.08656140886456208, b=6.986431801923841e-18, loss=0.8420305102180388
現在訓練：rmsprop
迭代=400, w=0.4043829999183758, b=-0.0004999949978831347, loss=0.3914575288307122
迭代=800, w=0.7935444182515551, b=-0.0004999949999999743, loss=0.1128150198997715
迭代=1200, w=0.9554709266898187, b=-0.0004999950000000075, loss=0.0861245032159488
迭代=1600, w=0.9554686209571135, b=-0.0004999950000000277, loss=0.08612450529503987
迭代=2000, w=0.9554686209571139, b=-0.0004999950000000288, loss=0.08612450529503986
現在訓練：adam
迭代=400, w=0.8056650383811171, b=1.63181198225288e-12, loss=0.10890608981150654
迭代=800, w=0.9321025003132031, b=4.6894193638234624e-17, loss=0.08669940059639455
迭代=1200, w=0.9534715992620724, b=5.0415690817968394e-17, loss=0.08613031897259907
迭代=1600, w=0.9558159496338102, b=5.471756553411393e-17, loss=0.08612402897687214
迭代=2000, w=0.9559638821331627, b=3.192854864709221e-17, loss=0.0861240053278867
```



優化器	課程特點	你的實驗觀察
SGD	固定學習率，易震盪，收斂慢	（請填寫觀察）
Adagrad	自動調整學習率，後期可能停滯	（請填寫觀察）
RMSProp	指數移動平均，更穩定的自適應	（請填寫觀察）
Momentum	加入慣性/動量，加速收斂	（請填寫觀察）
Adam	Momentum + RMSProp + 偏差修正，最推薦	（請填寫觀察）

SGD、Momentum 、RMSProp、Adam:回歸曲線中貼近資料，在 loos 曲線中也都有下降，Adam 的 Loss 曲線下降幅度較小是沒有問題的，只要有再下降都正常，而在最終的 Loss 結果都相差不遠，所以三種優化器效果皆差不多在多數場合中應該自由選用。

Adagrad:回歸曲線相離資料太遠，就算 Loss 有在下降，也不表示他效果，因為本來就是偏離資料上做訓練，所以效果不好。

(五) 心得

在觀察不同學習率下我發現，有些時候在資料不多時學習率應該要適中，在大筆資料時學習率應該要小一點，已讓它慢慢學習，但是工程實務上這都沒有絕對，所以才需要模型微調，而不同優化器間，因為帶入公式的數值優先順序不同，有時候會有不同成果，還是得依照專案做調整選用優化器。